

Simulations with nlmixr2 rxode2

R/Pharma 2026

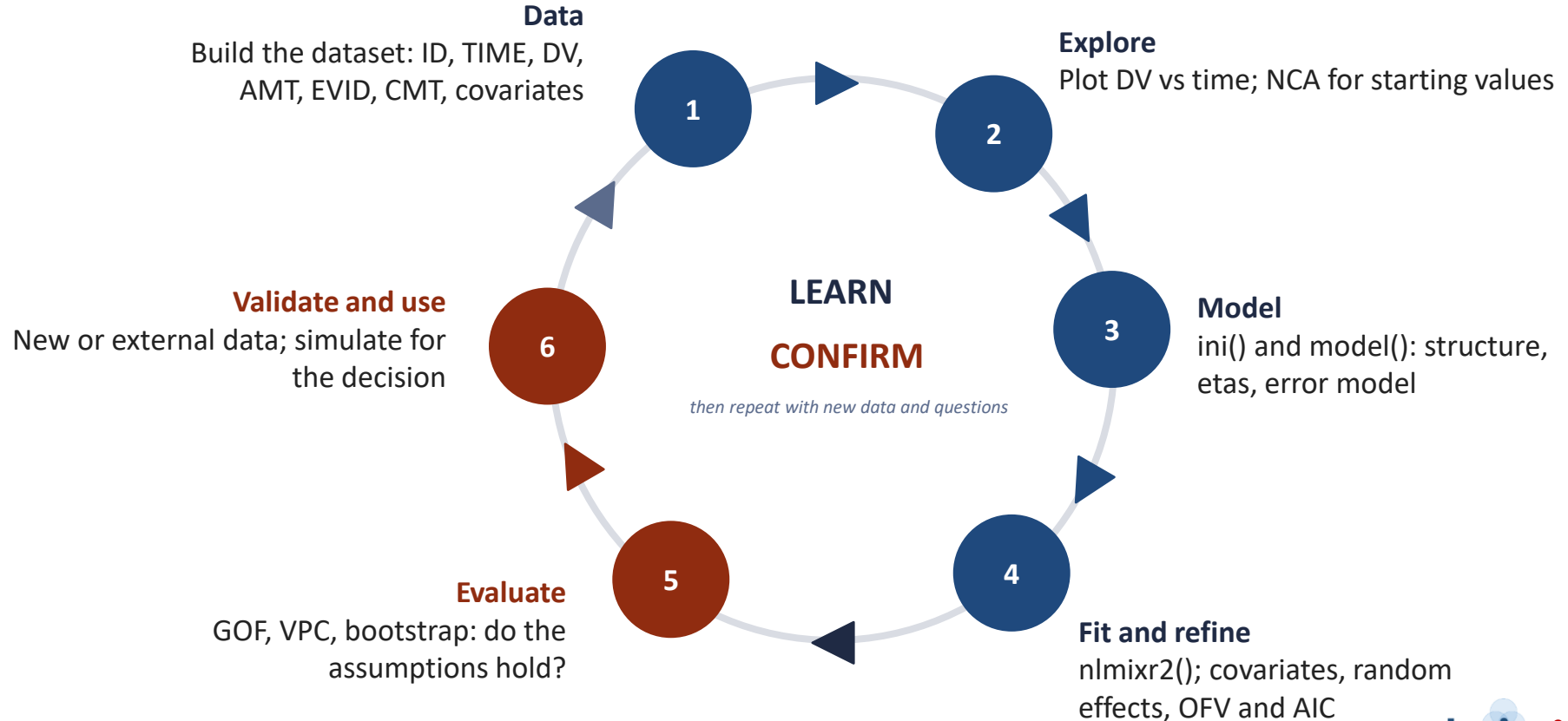
Anne Keunecke, PhD

On behalf of the **nlmixr** development team:

Matt Fidler, Bill Denney, John Harrold, Richard Hooijmaijers, Mutaz Jaber, Anne Keunecke, Theo Papathanasiou, Rik Schoemaker, Max Taubert, Mirjam Trame, Justin Wilkins, Brendan Bender



The learn-confirm cycle in nlmixr2 – from data to model in a single environment



rxode2 simulation software in R

- **rxode2** is pharmacometric simulation software using ordinary differential equations (ODEs) as an open-source R package
- Rewritten by Matt Fidler after Wenping Wang's original RxODE package, available on CRAN^a and GitHub^b, and described in a tutorial in CPT:PSP^c and with online documentation^d
- Simulation of ODEs was already possible in R (using deSolve), but was slow and virtually impossible to code with flexible dosing history
- **rxode2** has rapid execution due to compilation in C
- **rxode2** allows fully flexible dosing history
- Stable and mature software for Windows, OS X (in docker), Linux
- Requires external compilers (provided by Rtools on Windows)

[a] CRAN: <https://cran.r-project.org/web/packages/rxode2>

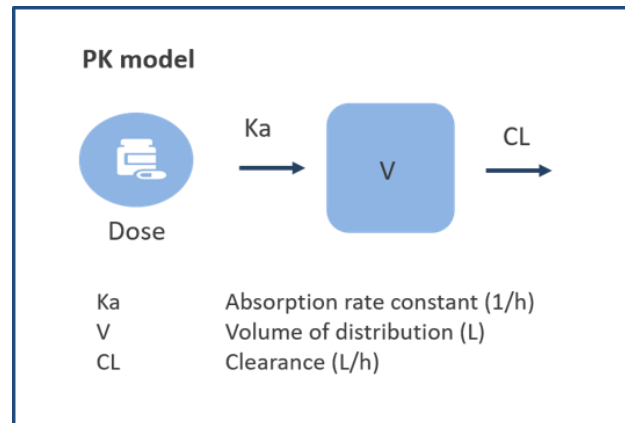
[b] GitHub: <https://github.com/nlmixr2/rxode2>

[c] Wang W et al. CPT:PSP (2016) 5, 3–10.

[d] RxODE packagedown: <https://nlmixr2.github.io/rxode2>

What do you need to perform a simulation

- Model
 - nlmixr2 (the estimation layer) on RxODE
 - The model syntax is essentially the same as in rxode2, extended with elements such as ini() blocks for initial estimates and model() blocks that include the residual error specification.
 - The fitted nlmixr2 model can directly be used for simulations with rxode2 functions, for example to run simulations with the estimated parameters and uncertainty.
 - Updating or adjustments of the models work the same way.



```
One.comp.KA.solved <- function() {  
  ini({  
    # Where initial conditions/variables are specified  
    lka <- log(1.15) # Log ka (1/h)  
    lcl <- log(0.135) # Log CL (L/h)  
    lv <- log(8) # Log V (L)  
    prop.err <- 0.15 # proportional error (SD/mean)  
    add.err <- 0.6 # additive error (mg/L)  
    eta.ka ~ 0.5  
    eta.cl ~ 0.1  
    eta.v ~ 0.1  
  })  
  model({  
    # Where the model is specified  
    cl <- exp(lcl + eta.cl)  
    v <- exp(lv + eta.v)  
    ka <- exp(lka + eta.ka)  
    ## solved system example  
    ## where residual error is assumed to follow proportional and  
    additive error  
    linCmt() ~ prop(prop.err) + add(add.err)  
  })  
}
```

What do you need to perform a simulation

- Typical structure of a pharmacometric dataset

- Event types

- Each row represents a single event for a patient (a dose given or an observation recorded).
 - Columns generally answer, in order:

ID	TIME	AMT	DV	EVID	WT	AGE	SEX	SPARSE
1	0	100	0	1	66.7	50	1	0
1	0.5	0	0	0	66.7	50	1	0
1	1	0	1.9	0	66.7	50	1	0
1	2	0	3.3	0	66.7	50	1	0
1	3	0	6.6	0	66.7	50	1	0
1	6	0	9.1	0	66.7	50	1	0



- Minimal required columns

- ID - unique subject identifier
 - TIME - continuous and relative to the first event within a subject
 - AMT - amount of drug administered
 - DV - dependent variable.
 - EVID - event identifier, 0 = Observation, 1 = Dose, 2 = Other
- Details: <https://nlmixr2.github.io/rxode2/articles/rxode2-event-types.html>

```
## create an empty event table that stores  
## both dosing and sampling information :  
ev <- eventTable()  
## add a dose to the event table:  
ev$add.dosing(dose = 500) # mg  
## add time points to the event table where  
## concentrations will be simulated  
## these actions are cumulative  
ev$add.sampling(seq(0, 120, 0.1))
```